

Image Processing

Assignment – Edge Detection Using the Sobel Filter

Part A – Manual Computation

Alphin Joseph Alex
15 S2 ME
Page No. _____
Date / /

Edge Detection Using Sobel Filter

Given input image size : 5×5

10	10	10	80	80
10	10	10	80	80
10	10	10	80	80
10	10	10	80	80
10	10	10	80	80

Sobel $X = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$

1) Formula = $N - k + 1$ [more specifically $\frac{N - k + 2P}{s} + 1$]

Output Size = $\frac{5 - 3 + 2(0)}{1} + 1 = 3$

Output size = 3×3

$N \rightarrow$ input image size
 $k \rightarrow$ Filter size
 $P \rightarrow$ padding
 $s \rightarrow$ stride

2) First Output (top-left)

Image Patch

10	10	10
10	10	10
10	10	10

Sobel X kernel

-1	0	1
-2	0	2
-1	0	1

Page No.

Date

10	10	10	80	80
10	10	10	80	80
10	10	10	80	80
10	10	10	80	80
10	10	10	80	80

That box in the matrix (image size matrix) shows the kernel placement.

Output =	10 ⁻¹	10 ⁰	10 ¹
	10 ⁻²	10 ⁰	10 ²
	10 ⁻¹	10 ⁰	10 ¹

$$\begin{aligned}
 &= 10(-1) + 10(0) + 10(1) + \\
 &\quad 10(-2) + 10(0) + 10(2) + \\
 &\quad 10(-1) + 10(0) + 10(1) \\
 &= -10 + 0 + 10 - 20 + 0 + 20 - 10 + 0 + 10 \\
 &= 0
 \end{aligned}$$

Convolution = 0

$$\begin{aligned}
 3) \quad \begin{bmatrix} 10 & 10 & 80 \\ 10 & 10 & 80 \\ 10 & 10 & 80 \end{bmatrix} &= (-1)(10) + (1)(80) + (-2)(10) + (0)(10) + \\
 &\quad (2)(80) + (-1)(10) + (0)(10) + (1)(80) \\
 &= -10 + 80 - 20 + 160 - 10 + 80 \\
 &= 280
 \end{aligned}$$

Page No. _____

Date / /

$$\begin{bmatrix} 10 & 80 & 80 \\ 10 & 80 & 80 \\ 10 & 80 & 80 \end{bmatrix} = (-1)(10) + (0)(80) + (1)(80) + (-2)(10) + (0)(80) + (2)(80) + (-1)(10) + (0)(80) + (1)(80) \\ = -10 + 80 - 20 + 160 - 10 + 80 \\ = 280$$

$$\begin{bmatrix} 10 & 10 & 10 \\ 10 & 10 & 10 \\ 10 & 10 & 10 \end{bmatrix} = 0 \quad \begin{bmatrix} 10 & 10 & 80 \\ 10 & 10 & 80 \\ 10 & 10 & 80 \end{bmatrix} = 280 \quad \begin{bmatrix} 10 & 80 & 80 \\ 10 & 80 & 80 \\ 10 & 80 & 80 \end{bmatrix} = 280$$

$$\begin{bmatrix} 10 & 10 & 10 \\ 10 & 10 & 10 \\ 10 & 10 & 10 \end{bmatrix} = 0 \quad \begin{bmatrix} 10 & 10 & 80 \\ 10 & 10 & 80 \\ 10 & 10 & 80 \end{bmatrix} = 280 \quad \begin{bmatrix} 10 & 80 & 80 \\ 10 & 80 & 80 \\ 10 & 80 & 80 \end{bmatrix} = 280$$

3) Complete output matrix = $\begin{bmatrix} 0 & 280 & 280 \\ 0 & 280 & 280 \\ 0 & 280 & 280 \end{bmatrix}$

e) Analysis

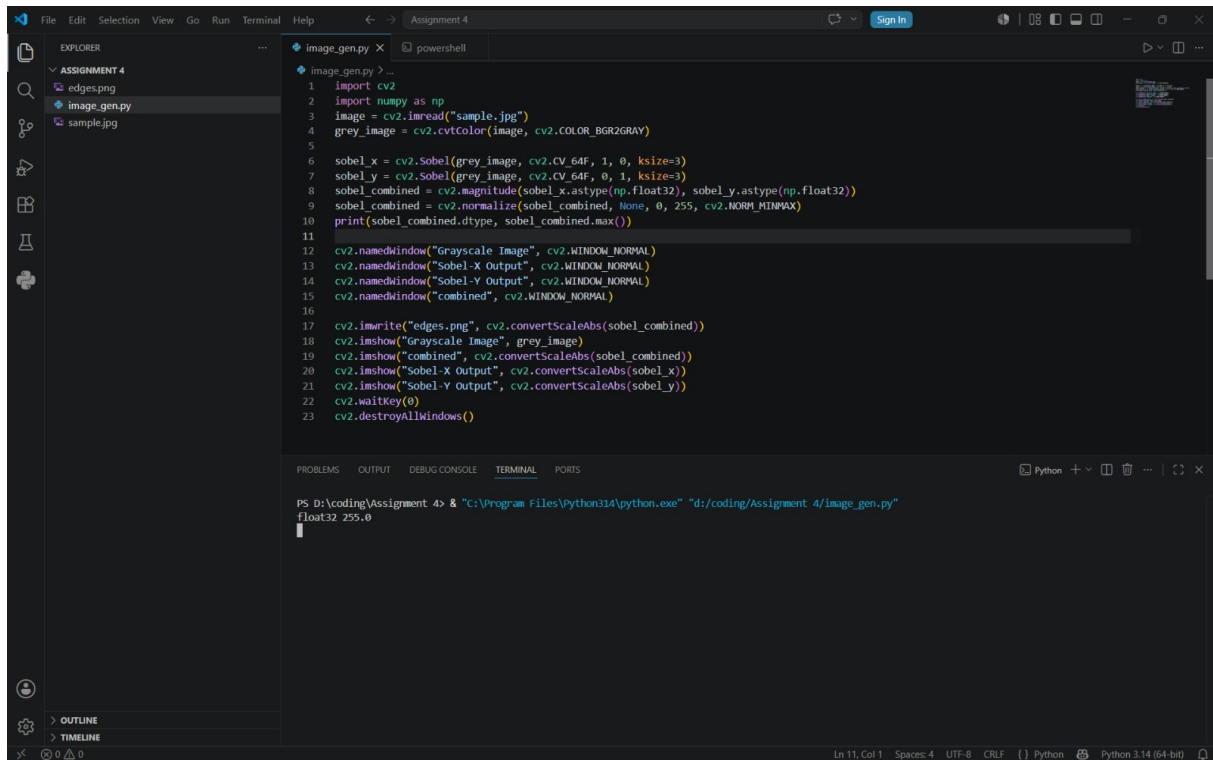
- The large values (280) indicate the presence of a vertical edge
- The edge occurs where the pixel values change from 10 to 80
- Homogenous regions produce values close to zero because there is little or no intensity change between neighbouring pixels
- The Sobel operator responds strongly only where there is a sudden change in intensity

Part B – Programming Using OpenCV

Python Source Code

```
import cv2
import numpy as np
image = cv2.imread("sample.jpg")
grey_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
sobel_x = cv2.Sobel(grey_image, cv2.CV_64F, 1, 0, ksize=3)
sobel_y = cv2.Sobel(grey_image, cv2.CV_64F, 0, 1, ksize=3)
sobel_combined = cv2.magnitude(sobel_x.astype(np.float32), sobel_y.astype(np.float32))
sobel_combined = cv2.normalize(sobel_combined, None, 0, 255, cv2.NORM_MINMAX)
print(sobel_combined.dtype, sobel_combined.max())
cv2.namedWindow("Grayscale Image", cv2.WINDOW_NORMAL)
cv2.namedWindow("Sobel-X Output", cv2.WINDOW_NORMAL)
cv2.namedWindow("Sobel-Y Output", cv2.WINDOW_NORMAL)
cv2.namedWindow("combined", cv2.WINDOW_NORMAL)
cv2.imwrite("edges.png", cv2.convertScaleAbs(sobel_combined))
cv2.imshow("Grayscale Image", grey_image)
cv2.imshow("combined", cv2.convertScaleAbs(sobel_combined))
cv2.imshow("Sobel-X Output", cv2.convertScaleAbs(sobel_x))
cv2.imshow("Sobel-Y Output", cv2.convertScaleAbs(sobel_y))
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Code Screenshot (VS Code)



```
1 import cv2
2 import numpy as np
3 image = cv2.imread("sample.jpg")
4 grey_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
5
6 sobel_x = cv2.Sobel(grey_image, cv2.CV_64F, 1, 0, ksize=3)
7 sobel_y = cv2.Sobel(grey_image, cv2.CV_64F, 0, 1, ksize=3)
8 sobel_combined = cv2.magnitude(sobel_x.astype(np.float32), sobel_y.astype(np.float32))
9 sobel_combined = cv2.normalize(sobel_combined, None, 0, 255, cv2.NORM_MINMAX)
10 print(sobel_combined.dtype, sobel_combined.max())
11
12 cv2.namedWindow("Grayscale Image", cv2.WINDOW_NORMAL)
13 cv2.namedWindow("Sobel-X Output", cv2.WINDOW_NORMAL)
14 cv2.namedWindow("Sobel-Y Output", cv2.WINDOW_NORMAL)
15 cv2.namedWindow("combined", cv2.WINDOW_NORMAL)
16
17 cv2.imwrite("edges.png", cv2.convertScaleAbs(sobel_combined))
18 cv2.imshow("Grayscale Image", grey_image)
19 cv2.imshow("combined", cv2.convertScaleAbs(sobel_combined))
20 cv2.imshow("Sobel-X Output", cv2.convertScaleAbs(sobel_x))
21 cv2.imshow("Sobel-Y Output", cv2.convertScaleAbs(sobel_y))
22 cv2.waitKey(0)
23 cv2.destroyAllWindows()
```

PS D:\coding\Assignment 4> & "C:\Program Files\Python314\python.exe" "d:/coding/Assignment 4/image_gen.py"
float32 255.0

Figure 0: Python source code in VS Code

Screenshots – Program Output Windows

Original Image (sample.jpg)



Figure 1: Original color input image (sample.jpg)

Grayscale Image



Figure 2: Image after BGR $\rightarrow$ Grayscale conversion

Sobel-X Output

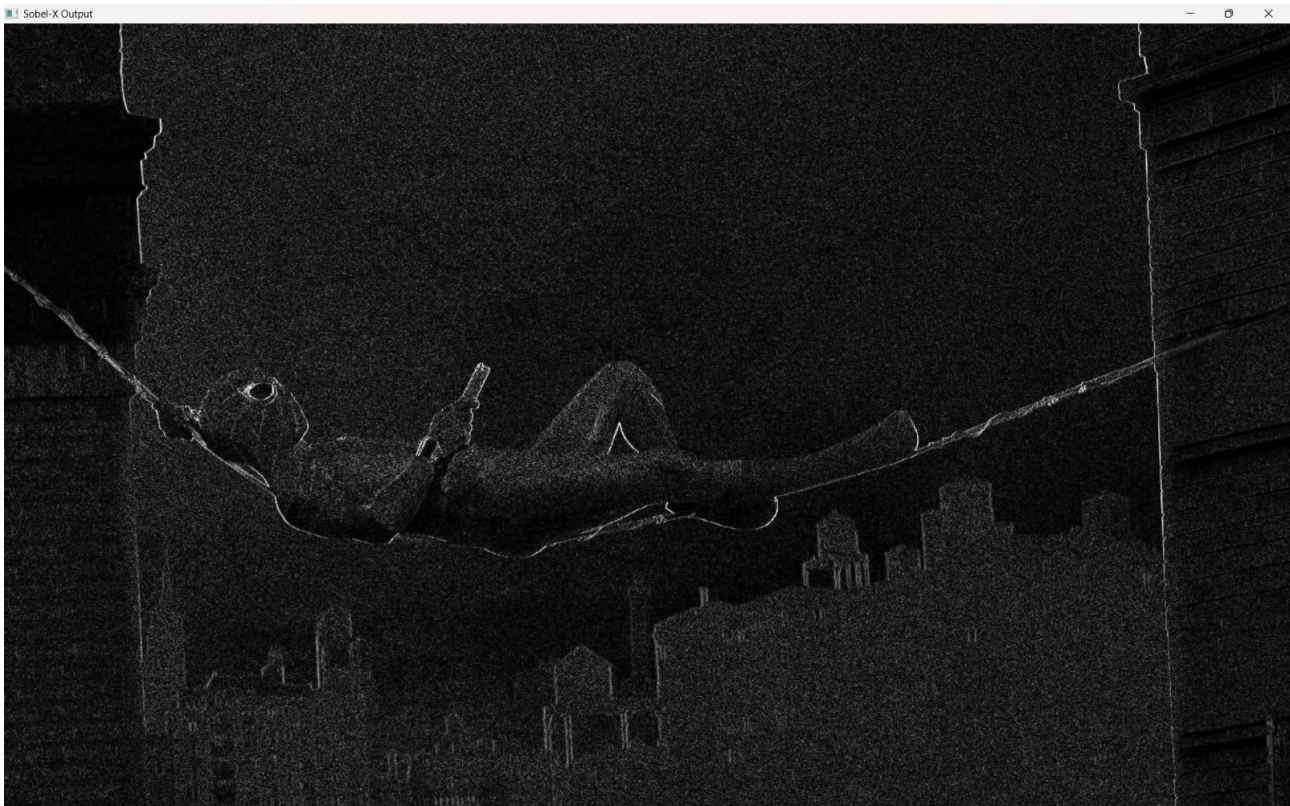


Figure 3: Sobel-X operator output – detects vertical edges

Sobel-Y Output



Figure 4: Sobel-Y operator output – detects horizontal edges

Final Edge-Detected Output (edges.png)



Figure 5: Combined gradient magnitude, normalized to [0-255]